

Coasterbot

Phasenpaper M0



Dennis Borutta, B. Eng	dennis.borutta@outlook.com
Stefan Etringer, B. Eng	mail@stefan.works
Gereon Such, M. Sc.	gereonsuch@gmail.com

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	20.07.2026	Such	Erstfassung des Phasenpapiers M0
2	21.07.2026	Etringer	Literaturrecherche zu Algorithmen ergänzt
3	21.07.2026	Borutta, Such	Literaturrecherche und Einleitungstext ergänzt; Coasterbot-Abbildung eingefügt; Timeline und Projektzeitraum angepasst
4	22.07.2026	Such	Versionshistorie ergänzt
5	23.07.2026	Borutta	Projektsteckbrief in Tabelle umgewandelt. Do- kumentenstruktur angepasst.

Inhaltsverzeichnis

1	Projektübersicht	4
2	Projektorganisation	5
3	Projektplanung	5
3.1	Projektstruktur	5
3.2	Netzplan	7
3.3	Zeitplan	9
4	Stand der Forschung: Mobile Roboter	11
5	Stand der Forschung: Algorithmen	11
5.1	Klassische Graphalgorithmen	11
5.2	Heuristische Verfahren	12
5.3	Dynamische Pfadplanung	12
5.4	Kollisionsvermeidung	13
5.5	SLAM und Lokalisierung	13
5.6	Manipulationsplanung	14
5.7	Inertialnavigation	14
6	Literaturverzeichnis	16

1 Projektübersicht

Kategorie	Inhalt
Projektname	Coasterbot – Entwicklung eines Prototyps
Projektzeitraum	Beginn: 15.07.2026 Abschluss: 07.11.2026
Projektziel	Entwicklung eines Prototyps eines Coasterbots: ein handteller-großer Roboter, der selbstständig auf einem Tisch navigiert, dort Getränkeuntersetzer in Richtung von Personen ausgibt und diese automatisiert wieder aufnimmt.
Weitere Ausbaustufen (Nicht im Fokus)	Automatisierte Tischreinigung, Aufnahme von Bestellungen, Initialisierung von Bezahlvorgängen
Motivation / Nutzen	• Entlastung des Bedienpersonals durch reduzierten Aufgabenbereich unterstützt damit den de seit Jahren bestehenden Fachkräftemangels in der Gastronomie [Deu26] • Alleinstellungsmerkmal und zusätzliches Attraktivitätsmerkmal für Bars/Restaurants • Potenziell mehr positive Bewertungen und verbesserte Online-Präsenz und damit Positiver Einfluss auf den Umsatz (Online-Präsenz als zunehmend relevanter Faktor bei der Restaurantwahl) [26b]
Definition of Done	• STL-Dateien und Schaltpläne zur Fertigung eines Prototypen sind fertiggestellt • Softwarearchitektur und -komponenten sind ausgearbeitet • Eine Simulation des Programmablaufes ist erstellt und validiert • Tests, die sich aus den Anforderungen ergeben, sind identifiziert und definiert
Out of Scope	• EMV-Betrachtungen • Kartierung der Umgebung • Einbindung in ein Netzwerk • Marketingkonzepte • Schwarmverhalten mehrerer Coasterbots

2 Projektorganisation

Das Projektteam besteht aus den drei Mitgliedern

Dennis Burutta	Leiter/Mitglied	dennis.borutta@outlook.com
Stefan Etringer	Admin/Mitglied	mail@stefan.works
Gereon Such	Mitglied	gereonsuch@gmail.com

Für die Projektorganisation und dokumentenbasierte Kommunikation im Team wird das Tool Notion <https://app.notion.com> genutzt. In diesem werden Projektphasen, Arbeitspakete, Fortschritt, Ergebnisse sowie Meilensteine protokolliert. Notion bietet hier vielfältige Planungsmöglichkeiten sowie Visualisierungen dazu. Eine Besonderheit dabei ist, dass Notion die Möglichkeit bietet kollaborativ an Dokumenten zu arbeiten und diese direkt an Arbeitspaketen zu knüpfen. So können die Arbeitsergebnisse und Referenzen strukturiert abgelegt werden.

Entwicklungsrelevante Dateien wie Source Code, CAD-Dateien für das Chassis oder Schaltungen werden in einem gemeinsamen Git Repository abgelegt.

Zur Remote Kommunikation wird eine Kombination aus Instant Messaging fürfristige Information und Absprachen sowie regelmäßigen Videocalls mit dem Tool Alfaview <https://alfaview.com> genutzt.

3 Projektplanung

Mit Hilfe der Tools Draw.io und Notion wird im Team die Projektplanung durchgeführt. Hierzu wird zur Ermittlung notwendiger Teilaufgaben und Arbeitspakete die Kreativmethode der Mind-Map angewandt. Daraus ergibt sich der Projektstrukturplan mit Arbeitspaketen gezeigt. Aus diesem wird anschließend eine Vorgangsliste erstellt, die in einem Netzplan überführt wird. Mit Hilfe des Netzplanes wird anschließend ein detaillierter Zeitplan in Notion erstellt.

3.1 Projektstruktur

Zur weiteren Bestimmung von erforderlichen Teilaufgaben und Arbeitspaketen wurde die Kreativmethode der Mind-Map angewandt. Die Ergebnisse hierzu sind in Abbildung 1 zu sehen. Daraus ergibt sich der Projektstrukturplan mit Arbeitspaketen wie in Abbildung 2 gezeigt. Hierbei wird ersichtlich, dass der Strang “Organisation” aus der Mind-Map nicht in dem Projektstrukturplan zu sehen ist. Dies wird begründet, da es sich dabei um fortlaufende Aufgaben handelt, die über die gesamte Projektlaufzeit hinweggeführt werden.

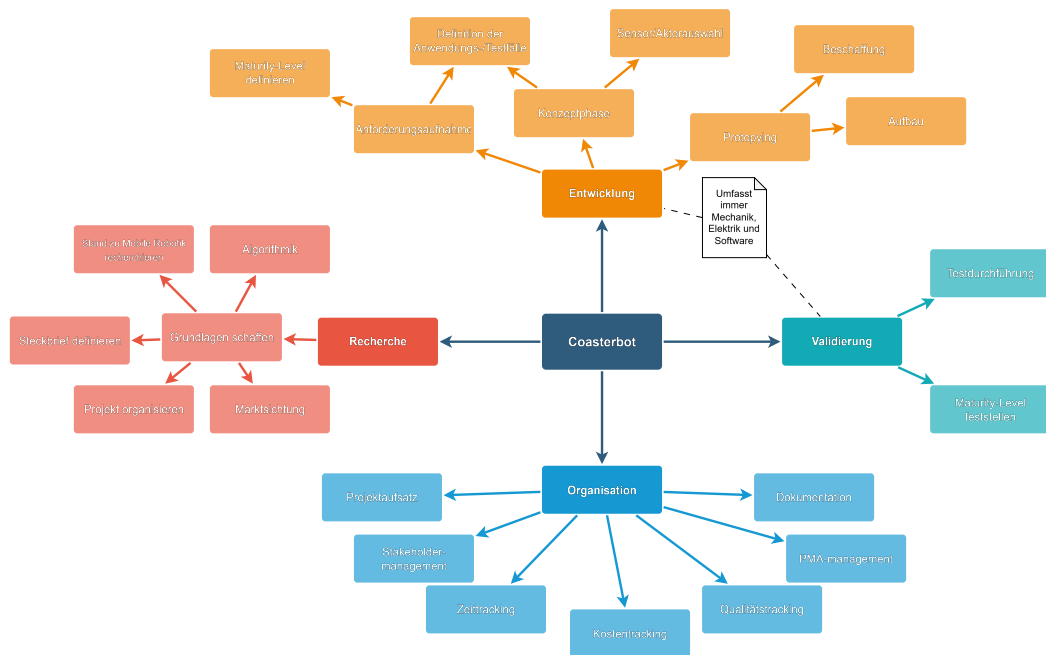


Abbildung 1: Mindmap zur Projektstruktur

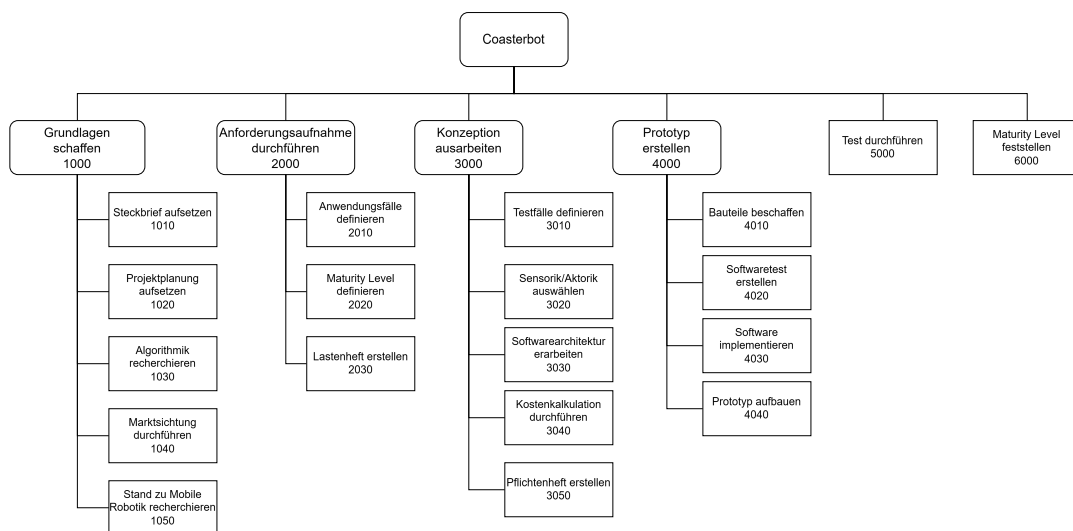


Abbildung 2: Projektstrukturplan

3.2 Netzplan

Mit dem Zeithorizont von 4 Monaten ergibt sich mit Gewichtung der jeweiligen Arbeitspakete nach Aufwand und Priorität der in Abbildung 3 dargestellte Netzplan nach DIN 69900. Aufgrund seiner Breite ist dieser auf einer eigenen Seite im Querformat abgebildet.

Netzplan Coasterbot — Endtermin 07.11.2026 (Endbericht), verdichtet

15.07. – 07.11.2026 · 83 Arbeitstage verfügbar · 82 AT verplant · 1 AT Endpuffer · Endbericht fertig 05.11.2026

Knoten: FAZ | Dauer | FEZ / AP-Nr: Arbeitspaket (Ressource) / SAZ | GP | SEZ — roter Rahmen/Kante = kritischer Pfad - gestrichelte Kante = Fast-Tracking (Vorlauf)

■ 1000 Recherche ■ 2000 Anforderungsaufnahme ■ 3000 Konzeption ■ 4000 Prototyp erstellen ■ 5000 Test durchführen ■ 6000 Maturity Level ■ 7000 Endbericht

Verdichtungsmaßnahmen (Plan lag 17 AT über Deadline):

M1 Langläufiger Beschaffung vor Pflanzzeitpunkt vorgezogen

M2 Prototypaufbau mit TM-A+TM-B gemeinsam (15–10 AT)

M3 Test mit TM-B+TM-A gemeinsam (12–8 AT) + Vorlauf zu SW

M4 Endbericht überlappt Maturity-Feststellung (Vorlauf 3 AT)

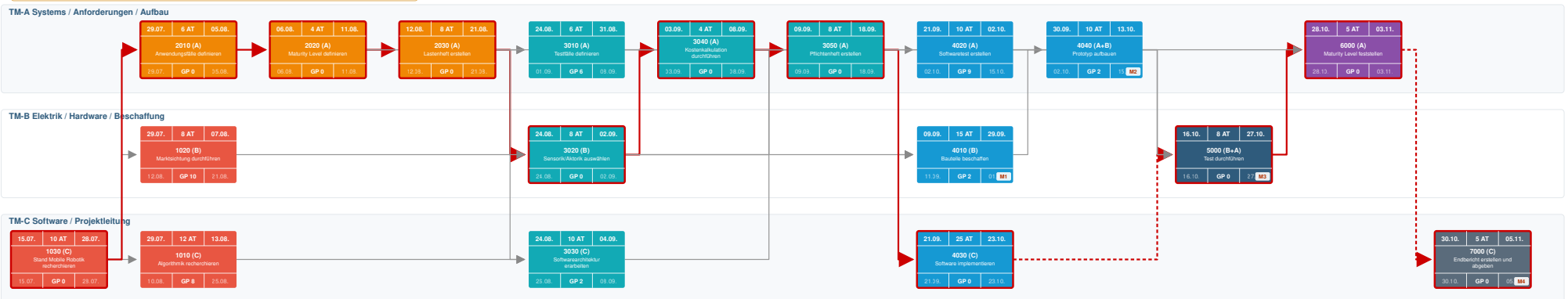


Abbildung 3: Netzplan

3.3 Zeitplan

Mit Hilfe des Netzplanes wird der Zeitplan detailliert. Dieser ist in Abbildung4 zu sehen. Aufgrund der Größe ist er auf einer eigen Seite im Querformat zu sehen und es sind nur Phase, Teilaufgaben und Meilensteine dargestellt.

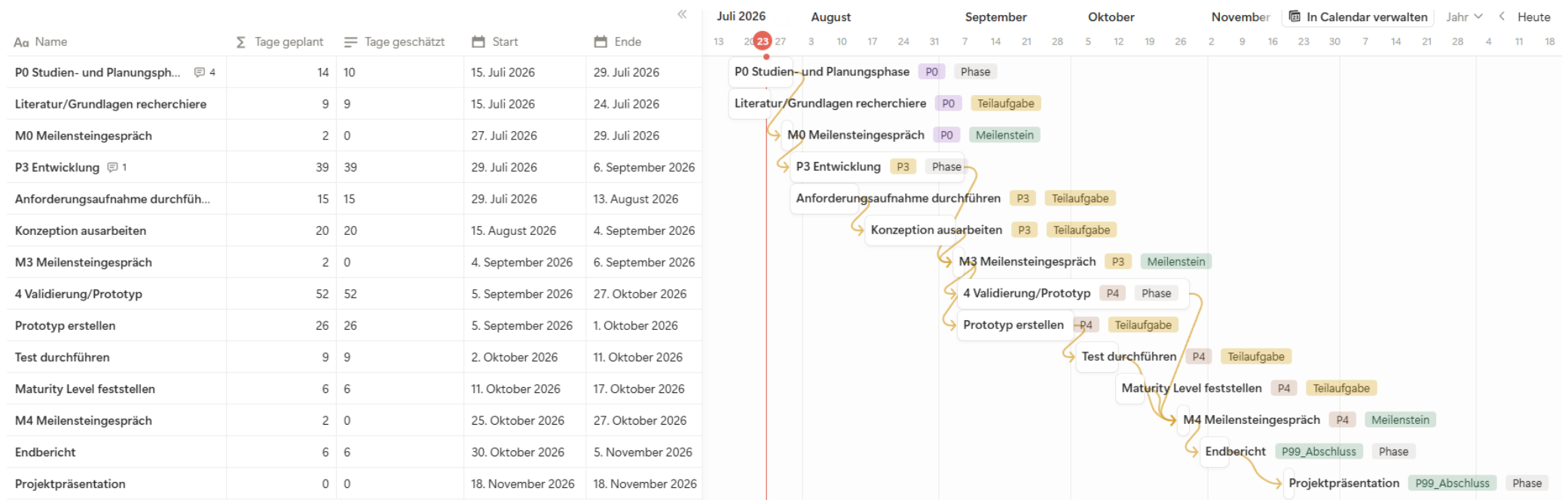


Abbildung 4: Zeitplan

4 Stand der Forschung: Mobile Roboter

Das Feld der mobilen Roboter ist bereits weit erforscht und es lassen sich eine Vielzahl an Arbeiten finden, die unterschiedliche Ansätze und Lösungen für die Navigation, Manipulation und Interaktion von Robotern in verschiedenen Umgebungen präsentieren. Eine übersichtliche Zusammenfassung relevanter Arbeiten ist in der Tabelle 2 im Anhang dargestellt. Diese Tabelle enthält Informationen zu den jeweiligen Roboter-Projekten. Diese werden als Grundlage zur Bauteilauswahl und Softwarearchitektur des Coasterbots dienen. Insbesondere die Arbeit von [Bal+26] kann als Grundlage dienen, da hier mehrere mobile Roboter Varianten vorgestellt und mit fertigen Stücklisten und CAD-Dateien veröffentlicht wurden. Damit bieten diese Arbeiten eine gute Grundlage für die Entwicklung des Coasterbots, so dass sich das Team auf die Softwarearchitektur, die Navigation des Roboters und die Hauptfunktion - das Ablegen und Aufnehmen von Getränkeuntersetzer - konzentrieren kann.

5 Stand der Forschung: Algorithmen

5.1 Klassische Graphalgorithmen

Die Grundlage vieler Verfahren zur autonomen Navigation bilden klassische Graphalgorithmen. Dabei wird die Umgebung als Graph modelliert, dessen Knoten diskrete Positionen oder Zustände repräsentieren, während Kanten mögliche Bewegungen zwischen diesen beschreiben. Ziel ist die Bestimmung eines optimalen Pfades hinsichtlich einer Kostenfunktion, beispielsweise der zurückgelegten Distanz, Fahrzeit oder des Energieverbrauchs.

Ein Vertreter ist der Dijkstra-Algorithmus, der den kürzesten Pfad in Graphen mit Kantengewichten berechnet. Der Algorithmus garantiert optimale Lösungen insofern die Kantengewichte nicht-negativ sind und stellt ein Referenzverfahren für die globale Pfadplanung in statischen Umgebungen dar [Dij59]. Seine mathematische Grundlage sowie Laufzeitanalyse werden ausführlich in "Introduction to Algorithms" von Cornet et al. beschrieben [Cor+26].

Neben Dijkstra existieren weitere grundlegende Suchverfahren, wie die Breitensuche (Breadth-First Search) und Tiefensuche (Depth-First Search, DFS)[Cor+26]. Während BFS optimale Lösungen in ungewichteten Graphen liefert, dient DFS vor allem der Strukturanalyse von Graphen und spielt in der Roboternavigation eine untergeordnete Rolle.

Für Anwendungen mit einer vollständig bekannten und statischen Umgebung stellen diese Algorithmen eine zuverlässige Grundlage der globalen Pfadplanung dar.

5.2 Heuristische Verfahren

Da klassische Graphalgorithmen bei großen Suchräumen einen hohen Rechenaufwand verursachen, werden in der mobilen Robotik häufig heuristische Suchverfahren eingesetzt. Ein bedeutendes Verfahren ist der heuristische A* Algorithmus, um die Suche gezielt in Richtung des Zielknotens zu lenken [Cor+26; HNR68a].

Die Bewertung eines Knotens erfolgt über

$$f(n) = g(n) + h(n) \quad (1)$$

Hierbei stellen $g(n)$ die wirklichen Kosten eines optimalen Pfades s nach n und $h(n)$ die wirklichen Kosten eines optimalen Pfades von n zu einem präferierten Zielknoten von n darstellt. Allgemein ist $h(n)$ eine Schätzung der verbleibenden Distanz zum Ziel [HNR68a]. Ist die Heuristik zulässig (admissible), garantiert A* ebenfalls optimale Lösungen, untersucht jedoch weniger Knoten als Dijkstra.

Für mobile Roboter kommen häufig euklidische oder Manhattan-Distanzen [Cor+26] als Heuristik zum Einsatz. Dadurch eignet sich A* insbesondere für Gitternetze (Occupancy Grids), wie sie in der Robotik genutzt werden[ME].

Aktuellere Varianten wie Weighted A* oder Anytime A* priorisieren kürzere Rechenzeiten gegenüber der optimalen Lösung, weshalb deren Anwendung geeignet in Echtzeitsystemen ist[ED09; HZ07].

5.3 Dynamische Pfadplanung

In realen Einsatzgebieten verändert sich die Umgebung häufig während der Fahrt. Statische Algorithmen müssten den gesamten Pfad erneut berechnen. Dies führt zu hohen Rechenzeiten.

Hier setzen Verfahren der dynamischen Pfadplanung an. Zu nennen sind unter anderem D* (Dynamic A*) [Ste] sowie dessen Weiterentwicklung D* Lite [KL02]. Beide Verfahren aktualisieren ausschließlich die von einer Umweltveränderung betroffenen Bereiche des Suchgraphen und vermeiden dadurch vollständige Neuberechnungen.

Ein weiteres relevantes Verfahren ist Lifelong Planning A* (LPA*), welches inkrementelle Änderungen effizient verarbeitet [KLF04]. Solche Algorithmen werden insbesondere in autonomen Fahrzeugen und mobilen Servicerobotern eingesetzt, deren Umgebungen durch Menschen oder bewegliche Objekte beeinflusst werden.

Für einen Coaster Bot wären dynamische Planungsverfahren dann relevant, wenn sich während der Navigation Gläser, Teller oder Personen im Arbeitsbereich befinden.

5.4 Kollisionsvermeidung

Während globale Pfadplanung den optimalen Weg bestimmen, übernimmt die Kollisionsvermeidung die kurzfristige Reaktion auf Hindernisse.

Ein Ansatz sind Potentialfeldmethoden, bei denen das Ziel eine anziehende Kraft und Hindernisse abstoßende Kräfte erzeugen. Diese Methoden sind einfach implementierbar, können jedoch in lokalen Minima stecken bleiben [Bit16].

In mobilen Robotersystemen wird u.a. der Dynamic Window Approach (DWA) verwendet, der die Bewegungsdynamik des Roboters selbst einbezieht [FBT97]. Diese Strategie ist grundsätzlich ein statischer Ansatz, der die Eigenschaften von Veränderungen in der Umgebung nicht berücksichtigt. Durch den Einsatz von Neuronalen Netzen kann DWA, durch kurze Observationen der sich verändernden Umgebung, die Bewegungen der Hindernisse modellieren und somit auf Veränderungen der Umwelt reagieren [DS24].

Für komplexe Konfigurationsräume kommen außerdem samplingbasierte Verfahren wie Rapidly-exploring Random Trees (RRT) [LaV98] sowie RRT* [Isl+] zum Einsatz. Während RRT schnell zulässige Pfade findet, verbessert RRT* diese schrittweise bis zum asymptotischen Optimum [Isl+].

Somit bildet die Kombination aus globalem Pfadplaner und lokaler Kollisionsvermeidung einen möglichen Ansatz zur Bewegung des Coaster Bots dar.

5.5 SLAM und Lokalisierung

Zur verbesserten Navigation kann ein Roboter seine eigene Position bestimmen und gleichzeitig eine Karte seiner Umgebung aufbauen. Dieses Problem wird als Simultaneous Localization and Mapping (SLAM) bezeichnet [18a].

Zu den klassischen Verfahren gehören Extended Kalman Filter (EKF-SLAM), FastSLAM auf Basis von Paketfiltern sowie graphbasierte SLAM Verfahren [DWW; 24; Mon+02].

Besonders graphbasierte Optimierungsverfahren haben sich aufgrund ihrer hohen Genauigkeit etabliert und bilden die Grundlage von Robotik-Frameworks wie ROS2 [26c].

Für den Coaster Bot hängt die Bedeutung von SLAM vom Einsatzszenario und den verwendeten Sensoren ab. Wird ausschließlich auf einem bekannten Tisch gearbeitet, genügt meist eine vorher definierte Karte. Soll das System autonom unbekannte Arbeitsbereiche erfassen oder sich frei im Raum bewegen können, ist ein SLAM Verfahren vorteilhaft.

5.6 Manipulationsplanung

Neben der Navigation stellt die Manipulationsplanung einen wesentlichen Bestandteil autonomer Serviceroboter dar. Sie umfasst die Planung und Steuerung von Greifbewegungen sowie das sichere Aufnehmen und Ablegen von Objekten. Hierbei müssen sowohl die Kinematik des Roboters als auch mögliche Kollisionen zwischen Greifer, Objekt und Umgebung berücksichtigt werden [Ber+; Zha+24].

Zur Bewegungsplanung von Roboterarmen werden häufig samplingbasierte Verfahren wie RRT [LaV98], RRT* [Isl+] oder Probabilistic Roadmaps (PRM) [Kav+96] eingesetzt. Moderne Robotik-Frameworks wie MoveIt 2 [25] kombinieren diese Planungsverfahren mit Kollisionsprüfungen und inverser Kinematik.

Für den Coaster Bot besteht die Manipulationsaufgabe darin, Getränkedeckel präzise aufzunehmen und an einer definierten Zielposition abzulegen. Neben einer genauen Positionierung des Roboters müssen dabei Greifstrategie, Objektorientierung sowie eine zuverlässige Ablage berücksichtigt werden.

5.7 Inertialnavigation

Die Inertialnavigation (Inertial Navigation System, INS) beschreibt Verfahren zur Bestimmung der Position, Orientierung und Bewegung eines mobilen Systems ausschließlich auf Grundlage von Trägheitssensoren. Im Gegensatz zu kamerabasierten oder laserbasierten Lokalisierungsmethoden ist die Inertialnavigation nicht auf externe Referenzpunkte oder eine Sichtverbindung zur Umgebung angewiesen. Dadurch eignet sie sich insbesondere für Anwendungen, in denen optische Sensoren nur eingeschränkt eingesetzt werden können oder zeitweise keine zuverlässigen Umgebungsinformationen zur Verfügung stehen [Gro13; LP17].

Ein Inertialnavigationssystem besteht zumeist aus einer Inertial Measurement Unit (IMU), die Beschleunigungssensoren (Accelerometer) und Drehratensensoren (Gyroskope) kombiniert. Moderne Systeme integrieren zusätzlich Magnetometer zur Bestimmung der absoluten Orientierung. Aus den gemessenen Beschleunigungen und Winkelgeschwindigkeiten werden durch numerische Integration Geschwindigkeit, Orientierung und Position des Roboters berechnet. Die mathematischen Grundlagen dieser Verfahren basieren auf den Bewegungsgleichungen der klassischen Mechanik sowie auf Verfahren zur Schätzung von Zustandsgrößen [Gro13; LP17].

Ein Problem der Inertialnavigation stellt der Drift dar. Geringe Messfehler der Sensoren werden durch die fortlaufende Integration akkumuliert und führen mit zunehmender Betriebsdauer zu wachsenden Positions- und Orientierungsfehlern. Aus diesem Grund wird eine reine Inertialnavigation in der mobilen Robotik nur selten dauerhaft eingesetzt. Stattdessen erfolgt eine Kombination mit externen Sensorsystemen wie LiDAR, Kameras oder Radencodern, um die Positionsschätzung regelmäßig zu korrigieren [Gro13; TBF05].

Zur Fusion verschiedener Sensordaten kommen häufig probabilistische Schätzverfahren wie der Kalman-Filter, der Extended Kalman Filter (EKF) [DWW] oder der Unscented Kalman Filter (UKF) [WV] zum Einsatz. Diese Verfahren kombinieren die Messdaten der IMU mit absoluten Positionsinformationen anderer Sensoren und ermöglichen dadurch eine robustere Lokalisierung. In aktuellen Robotersystemen wird zudem vermehrt auf graphbasierte Optimierungsverfahren und faktorgraphbasierte Ansätze zurückgegriffen, welche mehrere Sensordatenquellen gleichzeitig berücksichtigen und die Genauigkeit der Positionsbestimmung verbessern [TBF05;].

Die Inertialnavigation findet in zahlreichen Bereichen der Robotik Anwendung. Autonome Serviceroboter, fahrerlose Transportsysteme (AGV), Drohnen sowie mobile Manipulatoren nutzen IMUs zur Stabilisierung der Bewegung, zur Schätzung der Fahrzeuglage und zur kurzfristigen Positionsbestimmung. Besonders in Innenräumen, in denen satellitengestützte Navigationssysteme wie GPS nicht verfügbar sind, stellt die IMU eine wichtige Informationsquelle für die Bewegungsregelung dar [Far26]. Auch in IoT- und Embedded-Systemen werden aufgrund der geringen Baugröße, des niedrigen Energieverbrauchs und der geringen Kosten häufig kompakte Micro-Electro-Mechanical Systems Inertial Measurement Units (MEMS-IMUs) eingesetzt [; LP17].

Für den geplanten Coaster Bot kann die Inertialnavigation die globale Pfadplanung und sensorbasierte Navigation ergänzen. Während Verfahren wie A* oder Dijkstra den optimalen Fahrweg berechnen und LiDAR- oder Kamerasysteme Hindernisse erkennen, liefert die IMU kontinuierlich Informationen über Beschleunigung und Drehrichtung des Roboters [HNR68a; Cor+26]. Dadurch können Fahrbewegungen zwischen Sensormessungen präziser geschätzt sowie kurzfristige Positionsänderungen erfasst werden. Beim Anfahren definierter Ablage- oder Aufnahmepositionen verbessert die Orientierungsschätzung die Wiederholgenauigkeit des Systems. Die Kombination einer IMU mit Verfahren der globalen Pfadplanung entspricht dem in der mobilen Robotik etablierten Ansatz einer mehrstufigen Navigation [LP17;].

Da sich der Coaster Bot auf einer vergleichsweise kleinen Arbeitsfläche bewegt und Bewegungen präzise erfolgen sollten, ist eine alleinige Inertialnavigation aufgrund der Drift möglicherweise nicht ausreichend. Eine mögliche Kombination aus mehreren Sensoren wie IMU, Radencodern, einem LiDAR- oder kamerabasierten Lokalisierungssystem würden eine effizientere Positionsbestimmung erreichen.

6 Literaturverzeichnis

Bibliography

- [] *Springer Handbook of Robotics*. Cham, Switzerland: Springer International Publishing. ISBN: 978-3-319-32552-1.
- [18a] *OpenSLAM.org*. Mai 2018. URL: <https://openslam-org.github.io>.
- [18b] *Veter - robot for researchers*. [Online; accessed 20. Jul. 2026]. Dez. 2018. URL: <https://veterobot.com/index.html>.
- [20] *Build an Arduino Robot with your Science Buddies Bluebot Kit*. [Online; accessed 20. Jul. 2026]. Juli 2020. URL: https://www.sciencebuddies.org/science-fair-projects/project-ideas/Robotics_p033/robotics/edge-detecting-arduino-robot.
- [24] *EKF SLAM*. Juli 2024. URL: <https://ankit-khandelwal.com/slam.html>.
- [25] *MoveIt 2 Documentation — MoveIt Documentation: Rolling documentation*. Nov. 2025. URL: <https://moveit.picknik.ai/main/index.html>.
- [26a] *E-Puck*. [Online; accessed 20. Jul. 2026]. Jan. 2026. URL: <https://e-puck.gctronic.com>.
- [26b] *Repräsentative Umfrage: Digitale Gastronomie*. [Online; accessed 20. Jul. 2026]. Juli 2026. URL: <https://newsroom.metroag.de/de/news/repraesentative-umfrage-digitale-gastronomie?dt=20250221>.
- [26c] *ROS 2 Documentation — ROS 2 Documentation: Rolling documentation*. Juli 2026. URL: <https://docs.ros.org/en/rolling/index.html>.
- [Arv+19] Farshad Arvin u. a. „Mona: an Affordable Open-Source Mobile Robot for Education and Research“. In: *J. Intell. Rob. Syst.* 94.3 (Juni 2019), S. 761–775. ISSN: 1573-0409. DOI: 10.1007/s10846-018-0866-9.
- [Bal+26] Jose Balbuena u. a. „PlatROB: An open-source, modular, and low-cost hardware platform for mobile robotics and AI education“. In: *HardwareX* 25 (März 2026), e00747. ISSN: 2468-0672. DOI: 10.1016/j.ohx.2026.e00747.
- [Ber+] Dmitry Berenson u. a. „Manipulation planning on constraint manifolds“. In: *2009 IEEE International Conference on Robotics and Automation*. IEEE, S. 12–17. DOI: 10.1109/ROBOT.2009.5152399.
- [Bit16] Prof. Dr. O. Bittel. *Pfadplanung: Potentialfeldmethoden*. 2016. URL: https://www-home.htwg-konstanz.de/~bittel/msi_robo/Vorlesung/05_Planung_Potentialfeldmethoden.pdf.
- [Cor+26] Thomas H. Cormen u. a. *Introduction to Algorithms, fourth edition*. The MIT Press, Juli 2026. ISBN: 978-0-26204630-5.

- [Deu26] Deutscher Hotel- und Gaststättenverband e.V. (DEHOGA Bundesverband). *DEHOGA-Zahlenspiegel IV/2025*. Zahlenspiegel. Stand: 24. Februar 2026. Berlin: DEHOGA Bundesverband, 2026. URL: https://www.dehoga-bundesverband.de/fileadmin/Startseite/04_Zahlen___Fakten/DEHOGA-Zahlenspiegel_4._Quartal_2025.pdf.
- [Dij59] E. W. Dijkstra. „A note on two problems in connexion with graphs“. In: *Numer. Math.* 1.1 (Dez. 1959), S. 269–271. ISSN: 0029-599X. DOI: 10.1007/BF01386390.
- [DS24] Matej Dobrevski und Danijel Skočaj. „Dynamic Adaptive Dynamic Window Approach“. In: *IEEE Trans. Rob.* 40 (2024), S. 3068–3081. ISSN: 1552-3098. DOI: 10.1109/TR0.2024.3400932.
- [DWW] Zhou Dong, Yize Wu und Jun Wang. „Kalman Filter and Extended Kalman Filter: Theory, Numerical Applications and Performance Analysis“. In: *2025 5th International Symposium on Computer Technology and Information Science (ISCTIS)*. IEEE, S. 16–18. DOI: 10.1109/ISCTIS65944.2025.11065026.
- [ED09] Ruediger Ebdendt und Rolf Drechsler. „Weighted A* search - unifying view and application“. In: 173 (2009), S. 1310–1342. DOI: 10.1016/j.artint.2009.06.004.
- [Far26] Jay A. Farrell. *Aided Navigation: GPS with High Rate Sensors*. McGraw Hill, Juli 2026. ISBN: 978-0-07149329-1.
- [FBT97] D. Fox, W. Burgard und S. Thrun. „The dynamic window approach to collision avoidance“. In: *IEEE Rob. Autom. Mag.* 4.1 (März 1997), S. 23–33. DOI: 10.1109/100.580977.
- [Gon+12] J. Gonzalez-Gomez u. a. *A New Open Source 3D-Printable Mobile Robotic Platform for Education*. Jan. 2012. ISBN: 978-3-642-27481-7. DOI: 10.1007/978-3-642-27482-4_8.
- [Gro13] Paul Groves. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, Second Edition*. Artech, 2013. ISBN: 978-1-60807006-0. URL: <https://ieeexplore.ieee.org/document/9101092>.
- [HNR68a] Peter E. Hart, Nils J. Nilsson und Bertram Raphael. „A Formal Basis for the Heuristic Determination of Minimum Cost Paths“. In: *IEEE Trans. Syst. Sci. Cybernetics* 4.2 (Juli 1968), S. 100–107. DOI: 10.1109/TSSC.1968.300136.
- [HNR68b] Peter E. Hart, Nils J. Nilsson und Bertram Raphael. „A Formal Basis for the Heuristic Determination of Minimum Cost Paths“. In: *IEEE Trans. Syst. Sci. Cybernetics* 4.2 (Juli 1968), S. 100–107. DOI: 10.1109/TSSC.1968.300136.
- [HZ07] Eric A. Hansen und Rong Zhou. „Anytime heuristic search“. In: *J. Artif. Intell. Res.* 28.1 (März 2007), S. 267–297. ISSN: 1076-9757. DOI: 10.5555/1622591.1622599.

- [HZ17] Boru Diriba Hirpo und Prof. Wang Zhongmin. „Design and Control for Differential Drive Mobile Robot“. In: *International Journal of Engineering Research & Technology* 6.10 (Okt. 2017). ISSN: 2278-0181. DOI: 10.17577/IJERTV6IS100138.
- [Isl+] Fahad Islam u. a. „RRT*-Smart: Rapid convergence implementation of RRT* towards optimal solution“. In: *2012 IEEE International Conference on Mechatronics and Automation*. IEEE, S. 05–08. DOI: 10.1109/ICMA.2012.6284384.
- [Kar22] Suat Karakaya. „Mechanical design of a differential drive mobile robot platform“. In: *Global Journal of Computer Sciences Theory and Research* 12.1 (Apr. 2022), S. 01–11. ISSN: 2301-2587. DOI: 10.18844/gjcs.v12i1.6508.
- [Kav+96] L. E. Kavraki u. a. „Probabilistic roadmaps for path planning in high-dimensional configuration spaces“. In: *IEEE Trans. Robot. Automat.* 12.4 (Aug. 1996), S. 566–580. DOI: 10.1109/70.508439.
- [KL02] Sven Koenig und Maxim Likhachev. „D*lite“. In: *ACM Conferences*. American Association for Artificial Intelligence, Juli 2002, S. 476–483. DOI: 10.5555/777092.777167.
- [KLF04] Sven Koenig, Maxim Likhachev und David Furcy. „Lifelong planning A*“. In: *Artif. Intell.* 155.1-2 (Mai 2004), S. 93–146. ISSN: 0004-3702. DOI: 10.1016/j.artint.2003.12.001.
- [LaV06] Steven M. LaValle. *Planning Algorithms*. Cambridge, England, UK: Cambridge University Press, Mai 2006. ISBN: 978-0-52186205-9. DOI: 10.1017/CB09780511546877.
- [LaV98] Steven M. LaValle. „Rapidly-exploring random trees: A new tool for path planning“. In: *Technical Report (TR98-11), Computer Science Department* (1998). URL: <https://msl.cs.illinois.edu/~lavalle/papers/Lav98c.pdf>.
- [LP17] Kevin M. Lynch und Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, England, UK: Cambridge University Press, Juli 2017. DOI: 10.5555/3165183.
- [ME] H. Moravec und A. Elfes. „High resolution maps from wide angle sonar“. In: *Proceedings. 1985 IEEE International Conference on Robotics and Automation*. IEEE, S. 25–28. DOI: 10.1109/ROBOT.1985.1087316.
- [Mon+02] Michael Montemerlo u. a. „FastSLAM: a factored solution to the simultaneous localization and mapping problem“. In: *ACM Conferences*. American Association for Artificial Intelligence, Juli 2002, S. 593–598. DOI: 10.5555/777092.777184.

- [Plo12] Ploner Hospitality Consulting. *Studie über das europäische Gastgewerbe: Trends, Erfolgsfaktoren, Technologien. Mit Fokus auf Service und Funkboniersysteme*. Studie. Im Auftrag der Orderman GmbH durchgeführt. Salzburg, Österreich: Orderman GmbH, 2012. URL: <https://www.orderman.com/wp-content/uploads/Gastrostudie-DE-web.pdf>.
- [Rez+23] Paulo Rezeck u. a. „HeRo 2.0: a low-cost robot for swarm robotics research“. In: *Autonomous Robots* 47.4 (März 2023), S. 1–25. ISSN: 1573-7527. DOI: 10.1007/s10514-023-10100-0.
- [Ste] A. Stentz. „Optimal and efficient path planning for partially-known environments“. In: *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*. IEEE, S. 08–13. ISBN: 978-0-8186-5330. DOI: 10.1109/ROBOT.1994.351061.
- [TBF05] Sebastian Thrun, Wolfram Burgard und Dieter Fox. *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. Cambridge, MA, USA: The MIT Press, Sep. 2005. DOI: 10.5555/1121596.
- [WV] E. A. Wan und R. Van Der Merwe. „The unscented Kalman filter for nonlinear estimation“. In: *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373)*. IEEE, S. 04. ISBN: 978-0-7803-5800. DOI: 10.1109/ASSPCC.2000.882463.
- [Zha+24] Zhigen Zhao u. a. „A Survey of Optimization-based Task and Motion Planning: From Classical To Learning Approaches“. In: *arXiv* (Apr. 2024). DOI: 10.1109/TMECH.2024.3452509. eprint: 2404.02817.

Abbildungsverzeichnis

1	Mindmap zur Projektstruktur	6
2	Projektstrukturplan	6
3	Netzplan	8
4	Zeitplan	10

Tabelle 2: Vergleich mobiler Roboterplattformen aus der Literaturrecherche und Übertragbarkeit auf den Coasterbot

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
Mona [Arv+19]	Mechanischer Aufbau	Leichtbau-Chassis	~45 g Gesamtgewicht, kompakte Bauform	Bedingt – Coasterbot braucht mehr Nutzlast (Untersetzer-Magazin, ggf. Getränke)
	Mechanischer Aufbau	Räder & Radstand	Raddurchmesser 28 mm, Radstand 80 mm	Als Referenzgröße für Mini-Prototyp geeignet
	Aktor	2x DC-Motor mit Untersetzungsge-triebe	Direktuntersetzung 250:1, symmetrischer Differentialantrieb	Hoch – Differentialantrieb ist Standardlösung für Coasterbot-Fahrwerk
	Aktor	H-Brücken-Motortreiber	PWM-Ansteuerung je Motor, ca. 74 mW Leistungsbedarf	Hoch – Standard-Ansteuerung, direkt übertragbar
	Sensor	5x IR-Nahbereichssensor	35° Sensorabstand, Hinderniserkennung/Bumper-Funktion	Mittel – für Personenerkennung ggf. zu kurze Reichweite, aber gut für Tischkantennähe
	Sensor	Magnetischer Encoder	6-Pol-Magnetscheibe + Hall-Sensor pro Motor, 1500 Pulse/Radumdrehung	Hoch – für Odometrie/Positionsregelung des Coasterbots relevant
	Sensor	Spannungsteiler (Batterieüberwachung)	ADC-Messung der Akkuspannung	Hoch – einfache, günstige Lösung für Energiemanagement
	Steuerung	ATmega328 Mikrocontroller	8-bit AVR, 16 MHz, Arduino-kompatibel, 32 KB Flash	Hoch – ausreichend für Fahrsteuerung, ggf. zu schwach für Bilderkennung

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
Diff.-Drive-Plattform [Kar22]	Steuerung	Erweiterungsmodule (Raspberry Pi Zero, RF, WiFi, XBee)	Für höhere Rechenleistung/Kommunikation nachrüstbar	Hoch – Raspberry Pi als Rechen-Backbone für Vereinzelung/Erkennung denkbar
	Software	Arduino IDE (C/C++)	Offene Programmierumgebung	Hoch – einfacher Einstieg für Prototyp-Firmware
	Software	Stage-Simulator	2D-Simulation von Multi-Robot-Szenarien	Gering – eher für Schwarmszenarien, für Einzelroboter-DoD nicht zwingend nötig
	Mechanischer Aufbau	CAD-Chassis-Vollmodell	Solid-Modeling-Ansatz für Gesamtstruktur	Hoch – methodisches Vorbild für eigenes CAD/STL-Konzept
	Mechanischer Aufbau	Antriebsrad-Befestigung	3 Schrauben zwischen zwei gegenüberliegenden Scheiben, Bushings gegen Korrosion	Hoch – direkt anwendbares Konstruktionsdetail für Radanbindung
	Aktor	DC-Motor (radachsnah montiert)	Motoren innenliegend entlang der Radachse	Hoch – platzsparende Motoranordnung fürs Chassis
	Sensor	– (nicht spezifiziert)	Paper fokussiert rein mechanisch, keine Sensor-BOM	n/a
	Steuerung	– (nicht spezifiziert)	Keine Angaben zu Controller/Elektronik	n/a

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
	Software	CAD/Solid-Modeling-Software	Für mechanischen Entwurf verwendet	Hoch – Standardwerkzeug auch für STL-Erstellung im eigenen Projekt
Diff.-Drive Mobile Robot [HZ17]	Mechanischer Aufbau	3D-Chassis-Modell	CAD-Baugruppe von Antriebs- und Castor-Rad	Mittel – gutes Referenzmodell, aber generisch
	Aktor	2x 12V-DC-Motor mit Getriebekopf	Bürsten-DC-Motor, Übersetzung $n=3$	Hoch – Dimensionierungsbeispiel für Motorauswahl
	Sensor	Tachometer/Drehzahlgeber	$K_{tach} = 1,8$, zur Drehzahlrückführung	Hoch – für geschlossene Drehzahlregelung der Antriebsmotoren relevant
	Steuerung	PID-Regler	Kp/Ki/Kd-Tuning für Drehzahlregelung	Hoch – Regelkonzept direkt für Fahrsteuerung übernehmbar
	Software	MATLAB/Simulink	Modellierung & Simulation des Regelkreises	Hoch – geeignet für die im DoD geforderte Ablaufsimulation
Edge Detecting Arduino Robot [20]	Mechanischer Aufbau	Rad-Chassis (Kit-Basis)	Einfaches, kommerzielles Robot-Chassis	Gering – nur als didaktisches Beispiel, kein Custom-Design
	Aktor	DC-Antriebsmotoren	Standard-Radantrieb	Gering – funktional redundant zu anderen Quellen
	Sensor	IR-LED + IR-Fototransistor	Abwärts gerichtet, erkennt Tischkante/Cliff über fehlende Reflexion	Sehr hoch – Kernprinzip für die Tischkantenerkennung des Coasterbots

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
Miniskybot [Gon+12]	Steuerung	Arduino-Board	Einfacher Mikrocontroller zur	Hoch – ausreichend für einfache
	Software	Arduino IDE, Schwellenwertlogik	Schwellenwert-Auswertung Vergleich IR-Signal gegen festen Grenzwert	Kantenerkennungslogik Hoch – einfachster Einstieg für Cliff-Detection-Funktion
	Mechanischer Aufbau	3D-gedrucktes Chassis (9 Teile)	Front, Rear, 2 Räder, Batteriefach, -halter, 3-teiliges Castor-Rad	Sehr hoch – nahezu 1:1 übertragbares Konzept für STL-basierten Prototyp
	Mechanischer Aufbau	O-Ring-Reifen & M3-Verschraubung	Standard-O-Ringe als Bereifung, M3-Schrauben/Muttern	Hoch – kostengünstige, einfach zu beschaffende Lösung
	Aktor	2x modifizierter Hobby-Servomotor (Futaba 3003)	Für Dauerdrehung gehackt (Continuous Rotation)	Mittel – günstige Alternative zu DC-Motor+Getriebe, aber weniger präzise
	Sensor	2x SRF02-Ultraschallsensor	I2C-Bus, beliebig erweiterbar	Mittel – Ultraschall ggf. für Personen-/Hinderniserkennung nutzbar
	Steuerung	PIC16F876A Mikrocontroller (Skycube-Board)	8-bit, RS232-Programmierung, 4x AAA-Batterien	Mittel – funktional ausreichend, aber weniger verbreitet als Arduino/ATmega

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
Veter Robot [18b]	Software	SDCC Cross-Compiler, OpenS-CAD/FreeCAD, KiCad	Parametrisches CAD (Scripting) + offenes PCB-Design	Hoch – Open-Source-Toolchain für STL- und Schaltplan-Erstellung direkt nutzbar
	Mechanischer Aufbau	Dagu Rover 5 Kettenfahrwerk	Kommerzielles Tracked-Chassis als Basis	Gering – Kettenantrieb überdimensioniert für Coasterbot-Anwendung
	Mechanischer Aufbau	3D-gedruckter Aufbaukörper	Individuell anpassbarer Body für Sensorplatzierung	Hoch – Prinzip der modularen 3D-Druck-Karosserie übertragbar
	Aktor	Sparkfun Dual-Motortreiber TB6612FNG	Ansteuerung der Rover-5-Antriebsmotoren	Hoch – bewährter, kompakter Motortreiber-IC
	Aktor	Servomotor (rotierbarer Sensorkopf)	Für Ausrichtung von Kamera/Sonar	Mittel – evtl. für ausrichtbare Personendetektion nutzbar
	Sensor	3x Devantech SRF-02 + 1x SRF-08 Ultraschallsensor	Entfernungsmessung rund um den Roboter	Hoch – für Hindernis-/Tischkantenerkennung sowie Personenannäherung relevant
	Sensor	Devantech CMPS10 Kompass	Digitaler Kompass/Neigungssensor	Gering – für Coasterbot-Anwendung (Innenraum, kurze Strecken) meist verzichtbar

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
	Sensor	2x Kamera + GPS-Empfänger	Für Bildverarbeitung und Außennavigation	Gering – GPS out of scope (Kartierung/Navigation), Kamera evtl. für Personenerkennung
	Steuerung	BeagleBoard-xM (ARM+DSP)	Haupt-Recheneinheit für Bildverarbei-	Mittel – leistungsfähig, aber ggf. überdimensioniert;
	Steuerung	Pololu Power-Modul, Level-Shifter (Adafruit/Sparkfun)	tung/Kommunikation Spannungswandlung & Pegelanpassung zw. Logikebenen	Raspberry Pi als Alternative Hoch – Standardkomponenten für saubere Spannungsversorgung
	Software	On-/Offboard-Softwarestack (Git), Video-Streaming, OpenGL-Cockpit	Fernsteuerungs- und Telemetrie-Software	Gering – Fernsteuerung/Netzwerkanbindung ist laut DoD Out-of-Scope
e-puck [26a]	Mechanischer Aufbau	Zylindrisches Chassis	Ø 70 mm, Höhe 50 mm, Gewicht 200 g	Mittel – Formfaktor zu klein, aber Bauweise als Inspiration nutzbar
	Aktor	2x Schrittmotor mit Planetengetriebe	Präzise Positionierung, bis 13 cm/s	Mittel – Schrittmotoren bieten hohe Präzision, aber höhere Kosten als DC-Motor
	Sensor	8x IR-Näherungs-/Lichtsensoren (TCRT1000)	Rundum-Hinderniserkennung + Umgebungslicht	Hoch – umlaufende Sensoranordnung als Vorbild für 360°-Erkennung

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
	Sensor	3D-Beschleunigungssensor (IMU)	Lage-/Bewegungserkennung	Mittel – z. B. zur Erkennung von Kollisionen/Kippen nutzbar
	Sensor	VGA-Farbkamera (640x480)	Für Bildverarbeitung/Objekterkennung	Hoch – für optionale Bestellaufnahme/Personenerkennung relevant
	Sensor	3x omnidirektionales Mikrofon	Richtungserkennung von Audioquellen	Gering – für Coasterbot-Kernfunktion nicht erforderlich
	Steuerung	dsPIC-Mikrocontroller	30–60 MHz, DSP-Kern, 8 KB RAM, 144 KB Flash	Mittel – leistungsfähig, aber proprietäres Ökosystem ggü. Arduino/ATmega
	Steuerung	Bluetooth/RS232-Kommunikation	Kabellose Programmierung & Fernsteuerung	Gering – Netzwerkanbindung laut DoD Out-of-Scope, ggf. nur für Debugging
	Software	GNU GCC C-Compiler, Webots-Simulator	Offene Toolchain, 3D-Robotersimulation	Sehr hoch – Webots als möglicher Kandidat für die geforderte Programmablauf-Simulation
	Mechanischer Aufbau	4 modulare 3D-druckbare Baugruppen	Ackermann Drive Module (ADM), Omni-/Differential Drive Module (ODM/DDM), Control & Processing Module (CPM), 4-DoF Manipulator (AMM)	Sehr hoch – modulares Baukastenprinzip direkt auf Fahr-/Spender-/Greifmodul des Coasterbots übertragbar

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
	Mechanischer Aufbau	CPM-Chassis mit Wechselabdeckung	Trägt Jetson Nano & Akkus, abnehmbarer Deckel für Kamera/LiDAR-Nachrüstung	Hoch – Konzept für wartungsfreundliches, erweiterbares Elektronikgehäuse
	Aktor	Antriebsmotoren (Ackermann-Lenkung, Differential- & Mecanum-Antrieb)	Austauschbare Fahrmodule je nach Anforderung (10 kg Nutzlast, 25 cm Wenderadius)	Hoch – Mecanum-/Differentialantrieb als Alternativkonzept für präzises Rangieren am Tisch
	Aktor	4-DoF Manipulator-Modul (AMM)	Roboterarm-Modul, 450 g Nutzlast, für Greif-/Manipulationsaufgaben	Sehr hoch – direkt relevant für Aufnahme/Abgabe von Untersetzern bzw. Flüssigkeitsaufnahme
	Aktor	Motortreiber (auf Custom-PCB)	Ansteuerung der Antriebsmotoren je Fahrmodul	Hoch – Standardkomponente, gut dokumentiertes Referenzdesign
	Sensor	3x Ultraschallsensor (Hinderniserkennung)	Auf Custom-PCB des ODM/DDM integriert	Hoch – für Hindernis-/Personenerkennung rund um den Coasterbot nutzbar
	Sensor	IMU (Inertial Measurement Unit)	Lageerkennung/Orientierung des Fahrmoduls	Mittel – z. B. zur Erkennung von Kippen oder unebenem Untergrund

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
	Sensor	Motor-Encoder	Drehzahl- /Positionsrückführung je Antriebsmotor	Hoch – für Odometrie und präzises Anfahren am Tisch relevant
	Sensor	Erweiterungs-Mounts (LiDAR, Kamera)	Vorgesehene Montagepunkte, nicht standardmäßig verbaut	Mittel – Kartierung ist laut DoD Out-of-Scope, Kamera für Personen-/Objekterkennung interessant
	Steuerung	Arduino R3 Mikrocontroller (je Fahrmodul)	Steuert Motoren, Sensorik und I2C-Kommunikation je Modul	Hoch – bewährte, günstige Steuerungslösung für einzelne Subsysteme
	Steuerung	NVIDIA Jetson Nano (CPM)	Zentrale Recheneinheit für KI-/Bildverarbeitungs-Workloads, 40–100 min Laufzeit	Hoch – geeignet als Hauptrechner für Personen-/Objekterkennung im optionalen Ausbau
	Steuerung	Standardisierter I2C-Bus über DB9-Steckverbinder	Einheitliche Inter-Modul-Kommunikation zwischen allen Baugruppen	Sehr hoch – elegantes Konzept für saubere modulare Verkabelung (Fahr-/Spender-/Sensor-Modul)
	Steuerung	3x 18650 Li-Ion-Akku + Wippschalter/Relais	Energieversorgung inkl. einfacher Leistungsschaltung	Hoch – praxiserprobtes, günstiges Energiekonzept

Fortsetzung auf nächster Seite

Tabelle 2 – Fortsetzung von vorheriger Seite

Roboter / Paper	Kategorie	Komponente	Details / Spezifikation	Übertragbarkeit auf Coasterbot
	Software	Teleoperations- & Navigationssoftwa- re, Open-Source- Repository (OSF)	Für Fernsteuerung und autonome Navigation genutzt; alle Baupläne/Firmware frei verfügbar	Hoch – guter Fundus für Software-Architektur- Referenz sowie Lizenzmodell (CC-BY 4.0)